

300 MERN Stack Interview Questions & Answers

MongoDB • Express.js • React.js • Node.js

Hinglish mein samjhaya gaya — Freshers se Intermediate level ke liye

Compiled by Claude (Anthropic)

Table of Contents

JavaScript Fundamentals — 47 Questions

HTML & CSS Basics — 15 Questions

React.js — 60 Questions

Node.js — 44 Questions

Express.js — 39 Questions

MongoDB & Mongoose — 53 Questions

MERN Integration, Auth, Deployment & System Design — 42 Questions

Total: 300 Questions & Answers

JavaScript Fundamentals

Q1. JavaScript kya hai?

Ans: JavaScript ek high-level, interpreted, single-threaded programming language hai jo mainly web pages ko interactive banane ke liye use hoti hai. Ye client-side aur server-side (Node.js) dono jagah chal sakti hai.

Q2. var, let aur const mein kya difference hai?

Ans: var function-scoped होता है और hoisting के साथ undefined initialize होता है। let और const block-scoped होते हैं। let की value change हो सकती है, const की नहीं (reassign नहीं कर सकते), लेकिन const object/array के अंदर के properties mutate हो सकते हैं।

Q3. Hoisting kya hoti hai?

Ans: Hoisting JavaScript का वो behavior है जिसमें variable और function declarations को उनके scope के top पर move कर दिया जाता है compile time पर। var hoisted होके undefined होती है, let/const hoisted होती हैं लेकिन Temporal Dead Zone में रहती हैं।

Q4. Closures kya hote हैं?

Ans: Closure एक function है जो अपने outer (lexical) scope की variables को access कर सकता है, चाहे outer function execution खतम हो चुका हो। ये data privacy और function factories बनाने के लिए use होते हैं।

Q5. == और === में क्या फरक है?

Ans: == loose equality है जो comparison से पहले type coercion करता है। === strict equality है जो value और type दोनों check करता है बिना coercion के। Best practice है === use करना।

Q6. this keyword कैसे work करता है?

Ans: this का value function के call context पर depend करता है — regular function में caller object, arrow function में lexical (surrounding) scope से this लिया जाता है, और global scope में window/undefined (strict mode) होता है।

Q7. Arrow functions normal functions से कैसे different हैं?

Ans: Arrow functions अपना own 'this', 'arguments', या 'super' नहीं रखते — inherit करते हैं enclosing scope से। ये constructor के रूप में use नहीं हो सकते और इनमें concise syntax होता है।

Q8. Event loop kya है?

Ans: Event loop JavaScript का mechanism है जो call stack और callback/task queue को manage करता है, ताकि single-threaded JS asynchronous operations (I/O, timers) handle कर सके बिना blocking के।

Q9. Call stack और callback queue में क्या difference है?

Ans: Call stack synchronous code execute करता है LIFO order में। Callback queue में async callbacks (setTimeout, promises) wait करते हैं जब तक call stack empty ना हो, फिर event loop

unhe push karta hai stack mein.

Q10. Promises kya hote hain?

Ans: Promise ek object hai jo asynchronous operation ka eventual result (success ya failure) represent karta hai. Iski teen states hoti hain: pending, fulfilled, rejected. `.then()`, `.catch()`, `.finally()` se handle hota hai.

Q11. async/await kaise kaam karta hai?

Ans: `async/await` promises ke upar syntactic sugar hai jo asynchronous code ko synchronous jaisa dikhata hai. `async` function hamesha promise return karta hai, aur `await` promise resolve hone tak execution pause karta hai.

Q12. Callback hell kya hai aur kaise avoid karein?

Ans: Callback hell tab hota hai jab multiple nested callbacks ki wajah se code deeply indented aur unreadable ho jata hai. Ise Promises, `async/await`, ya named functions se avoid kiya ja sakta hai.

Q13. map(), filter(), aur reduce() mein kya difference hai?

Ans: `map()` har element transform karke naya array return karta hai. `filter()` condition ke basis par elements select karta hai. `reduce()` array ko single value mein accumulate karta hai.

Q14. Prototypal inheritance kya hai?

Ans: JavaScript objects ek dusre se prototype chain ke through inherit karte hain. Har object ka ek internal `[[Prototype]]` link hota hai jisse properties/methods lookup ho sakti hain agar khud mein na milein.

Q15. null aur undefined mein kya farak hai?

Ans: `undefined` ka matlab hai variable declare hua hai lekin value assign nahi hui. `null` ek intentional 'empty value' hai jo developer explicitly assign karta hai.

Q16. Event bubbling aur event capturing kya hai?

Ans: Event bubbling mein event target element se upar parent elements tak propagate hota hai. Event capturing iska ulta hai — root se target tak jata hai. `addEventListener` mein teesra param `true` karke capturing enable hoti hai.

Q17. Debouncing aur throttling mein kya difference hai?

Ans: Debouncing function call ko delay karta hai jab tak events aana band na ho jaye (last call ke baad execute). Throttling function ko fixed time interval mein sirf ek baar execute hone deta hai.

Q18. IIFE kya hota hai?

Ans: Immediately Invoked Function Expression ek function hai jo define hote hi turant execute ho jata hai — `(function(){...})()`. Ye variable scope isolate karne aur global namespace pollution avoid karne ke liye use hota hai.

Q19. Spread aur rest operator mein kya farak hai?

Ans: Spread `(...)` array/object ke elements ko expand karta hai (e.g. function calls, array copy). Rest operator multiple arguments ko ek array mein collect karta hai function parameters mein.

Q20. Destructuring kya hai?

Ans: Destructuring ek syntax hai jisse array ya object ki values ko directly variables mein extract kiya ja sakta hai, e.g. `const {name, age} = user;`

Q21. Template literals kya hote hain?

Ans: Template literals backticks (```) se likhe jaate hain aur `${}` syntax se variable interpolation aur multi-line strings allow karte hain.

Q22. JavaScript mein data types kitne hote hain?

Ans: Primitive types: String, Number, Boolean, Null, Undefined, Symbol, BigInt. Non-primitive: Object (jisme Array, Function bhi include hain).

Q23. Pass by value aur pass by reference kya hai?

Ans: Primitive types pass by value hote hain (copy pass hoti hai). Objects aur arrays pass by reference hote hain (memory reference pass hota hai).

Q24. Higher-order function kya hota hai?

Ans: Higher-order function wo function hai jo doosre function ko argument ke roop mein leta hai ya function return karta hai, jaise map, filter, reduce.

Q25. JavaScript mein 'strict mode' kya hai?

Ans: 'use strict' ek directive hai jo silent errors ko throw errors mein convert karta hai, unsafe actions prevent karta hai aur code ko safer/optimized banata hai.

Q26. Currying kya hai?

Ans: Currying ek technique hai jisme multiple arguments lene wale function ko sequence of functions mein convert kiya jata hai, har ek single argument leta hai.

Q27. JavaScript mein memory leak kaise hoti hai?

Ans: Memory leak tab hoti hai jab unused objects ki reference kahin reh jaati hai (global variables, forgotten timers, detached DOM nodes, closures) jisse garbage collector unhe clear nahi kar pata.

Q28. WeakMap aur Map mein kya difference hai?

Ans: Map mein keys koi bhi type ho sakti hain aur strong references hoti hain. WeakMap mein sirf objects keys ho sakte hain aur weak references hoti hain jo garbage collection allow karti hain.

Q29. Generator functions kya hote hain?

Ans: Generator functions (function*) yield keyword ka use karke execution pause/resume kar sakte hain, aur iterator return karte hain jisse values on-demand generate hoti hain.

Q30. Symbol data type kis liye use hota hai?

Ans: Symbol ek unique aur immutable primitive value hai jo mostly object properties ke unique identifiers banane ke liye use hota hai, taaki naming collisions avoid ho.

Q31. JSON.stringify aur JSON.parse kya karte hain?

Ans: `JSON.stringify()` JavaScript object ko JSON string mein convert karta hai. `JSON.parse()` JSON string ko wapas JavaScript object mein convert karta hai.

Q32. Array aur Object ko deep clone kaise karte hain?

Ans: `structuredClone(obj)`, `JSON.parse(JSON.stringify(obj))`, ya `Lodash` ki `cloneDeep()` function se deep clone kiya ja sakta hai.

Q33. Event delegation kya hai?

Ans: Event delegation ek pattern hai jisme parent element par ek hi event listener attach karke child elements ke events handle kiye jaate hain, event bubbling ka use karke.

Q34. Function.prototype.bind, call, aur apply mein kya difference hai?

Ans: `call()` aur `apply()` function ko turant invoke karte hain with specific 'this' — `call` comma-separated args leta hai, `apply` array leta hai. `bind()` ek naya function return karta hai fixed 'this' ke sath, jise baad mein call karte hain.

Q35. Optional chaining (?.) kya hai?

Ans: Optional chaining operator (`?.`) nested object properties ko safely access karne deta hai bina error throw kiye agar koi intermediate property null/undefined ho.

Q36. Nullish coalescing operator (??) kya hai?

Ans: `??` operator left operand agar null ya undefined ho tabhi right operand return karta hai, warna 0 ya "" jaisi falsy values ko bhi valid maanta hai (unlike `||`).

Q37. Set data structure kya hai?

Ans: Set ek collection hai jisme sirf unique values store hoti hain, koi duplicate allow nahi hota. Iterable hota hai aur insertion order maintain karta hai.

Q38. Array.prototype.forEach() aur map() mein kya difference hai?

Ans: `forEach()` sirf iterate karta hai aur undefined return karta hai (no chaining). `map()` naya array return karta hai transformed values ke sath, jo chain ho sakta hai.

Q39. Temporal Dead Zone (TDZ) kya hai?

Ans: TDZ wo zone hai block ke start se `let/const` declaration tak, jahan variable access karne par `ReferenceError` aata hai kyunki wo hoisted hai lekin initialize nahi hua.

Q40. Shallow copy aur deep copy mein kya farak hai?

Ans: Shallow copy sirf top-level properties copy karta hai (nested objects still reference share karte hain). Deep copy nested structures ko bhi completely independent copy karta hai.

Q41. Promise.all(), Promise.race(), aur Promise.allSettled() mein kya difference hai?

Ans: `Promise.all()` sab promises resolve hone par resolve hota hai (ek reject hua toh fail). `Promise.race()` sabse pehle settle hone wale ka result deta hai. `Promise.allSettled()` sab promises ka result deta hai chahe fulfilled ho ya rejected.

Q42. Microtasks aur macrotasks mein kya difference hai?

Ans: Microtasks (Promises, queueMicrotask) macrotasks (setTimeout, setInterval) se pehle execute hote hain event loop ke har cycle mein.

Q43. Type coercion kya hai?

Ans: Type coercion JavaScript ka automatic ya implicit conversion hai ek data type se dusre mein, jaise string ko number mein comparison ke time.

Q44. Array methods slice() aur splice() mein kya difference hai?

Ans: slice() original array ko change kiye bina ek portion return karta hai (non-mutating). splice() original array ko modify karta hai — elements add/remove karta hai (mutating).

Q45. ES6 mein classes kaise work karti hain?

Ans: ES6 classes prototype-based inheritance ke upar syntactic sugar hain. constructor(), methods, static methods, aur extends keyword se inheritance define hoti hai.

Q46. Immediately vs deferred script loading (async/defer) mein kya farak hai?

Ans: async script download hote hi execute hoti hai (parsing block karke), defer HTML parsing complete hone ke baad, order maintain karke execute hoti hai.

Q47. What is a pure function?

Ans: Pure function wo hai jo same input par hamesha same output deta hai aur koi side effect nahi karta (external state ko modify nahi karta).

HTML & CSS Basics

Q48. Semantic HTML kya hai?

Ans: Semantic HTML tags (header, footer, article, section, nav) apna meaning clearly convey karte hain, jo accessibility aur SEO improve karta hai compared to generic div/span.

Q49. CSS Box Model kya hai?

Ans: Box model mein har element content, padding, border, aur margin se bana hota hai jo element ka total size aur spacing decide karta hai.

Q50. Flexbox aur Grid mein kya difference hai?

Ans: Flexbox one-dimensional layout (row ya column) ke liye best hai. Grid two-dimensional layout (rows aur columns dono) ke liye design kiya gaya hai.

Q51. position: relative, absolute, fixed, sticky mein kya farak hai?

Ans: relative apni normal position se offset hota hai. absolute nearest positioned ancestor se relative hota hai. fixed viewport ke relative fixed rehta hai. sticky scroll ke basis par relative/fixed switch karta hai.

Q52. CSS specificity kaise calculate hoti hai?

Ans: Specificity inline styles > IDs > classes/attributes/pseudo-classes > elements ke order mein calculate hoti hai, higher specificity wala rule apply hota hai.

Q53. Responsive web design kaise achieve karte hain?

Ans: Media queries, flexible grid layouts, relative units (% , em , rem , vw/vh), aur flexible images ka use karke responsive design achieve hota hai.

Q54. z-index kya karta hai?

Ans: z-index positioned elements ki stacking order (kaunsa element upar dikhega) control karta hai, higher value upar render hoti hai.

Q55. CSS mein rem aur em mein kya difference hai?

Ans: em parent element ke font-size ke relative hota hai, rem root (html) element ke font-size ke relative hota hai.

Q56. localStorage, sessionStorage aur cookies mein kya difference hai?

Ans: localStorage data permanently browser mein store karta hai. sessionStorage tab close hone tak rehta hai. Cookies server ko bhi bheje jaate hain aur expiry set ho sakti hai.

Q57. Viewport meta tag kya karta hai?

Ans: <meta name='viewport'> mobile devices par page ko sahi tarah scale aur render karne ke liye control deta hai, responsive design ke liye zaroori hai.

Q58. CSS preprocessors (SASS/LESS) kyun use karte hain?

Ans: Ye variables, nesting, mixins, aur functions provide karte hain jo plain CSS likhna easier aur maintainable banata hai.

Q59. Media queries kaise likhte hain?

Ans: @media (max-width: 768px) { ... } jaise syntax se specific screen sizes ke liye CSS rules apply kiye jaate hain.

Q60. CSS animations aur transitions mein kya difference hai?

Ans: Transitions properties ke beech smooth change dikhate hain trigger hone par (hover, etc). Animations @keyframes ke through zyada complex, multi-step, aur automatic ho sakti hain.

Q61. Block, inline, aur inline-block elements mein kya farak hai?

Ans: Block elements full width lete hain aur new line se start hote hain. Inline elements sirf content jitni width lete hain. inline-block dono properties combine karta hai.

Q62. BEM naming convention kya hai?

Ans: BEM (Block Element Modifier) ek CSS naming methodology hai jo class names ko structured aur reusable banata hai, jaise .card__title--active.

React.js

Q63. React kya hai?

Ans: React ek open-source JavaScript library hai Facebook (Meta) dwara banaya gaya, jo component-based, declarative UI banane ke liye use hota hai, especially single-page applications ke liye.

Q64. Virtual DOM kya hai?

Ans: Virtual DOM ek lightweight in-memory representation hai real DOM ka. React changes ko pehle Virtual DOM par apply karta hai, phir diffing algorithm se minimum real DOM updates calculate karke perform karta hai.

Q65. React mein reconciliation kya hai?

Ans: Reconciliation wo process hai jisse React Virtual DOM trees ko compare (diff) karta hai purane aur naye state ke beech, aur sirf changed parts ko real DOM mein update karta hai.

Q66. JSX kya hai?

Ans: JSX (JavaScript XML) ek syntax extension hai jo HTML-like code JavaScript ke andar likhne deta hai. Babel isse `React.createElement()` calls mein transpile karta hai.

Q67. Functional components aur class components mein kya difference hai?

Ans: Functional components simple functions hain jo JSX return karte hain aur hooks use karte hain state/lifecycle ke liye. Class components ES6 classes hain jo state aur lifecycle methods (`componentDidMount` etc.) use karte hain. Modern React functional components + hooks recommend karta hai.

Q68. Props kya hote hain?

Ans: Props (properties) parent se child component ko data pass karne ka tarika hain. Ye read-only hote hain aur child unhe directly modify nahi kar sakta.

Q69. State kya hai aur props se kaise different hai?

Ans: State component ke andar manage hone wala mutable data hai jo change hone par re-render trigger karta hai. Props external hote hain aur parent se aate hain, immutable hote hain child ke perspective se.

Q70. useState hook kya karta hai?

Ans: `useState` functional component mein local state add karne ke liye use hota hai. Ye ek array return karta hai `[state, setState]` with current value aur updater function.

Q71. useEffect hook kya karta hai?

Ans: `useEffect` side effects handle karta hai (data fetching, subscriptions, DOM manipulation) functional components mein. Ye `componentDidMount`, `componentDidUpdate`, aur `componentWillUnmount` ka combination hai.

Q72. useEffect ka dependency array kaise kaam karta hai?

Ans: Empty array [] se effect sirf mount par chalta hai. Dependencies diye jaane par unke change hone par re-run hota hai. Array na dene par har render ke baad chalta hai.

Q73. useContext hook kya hai?

Ans: useContext Context API ke through data ko component tree mein deeply pass karne deta hai bina props drilling ke, jaise theme ya authenticated user info share karna.

Q74. useRef hook kya karta hai?

Ans: useRef ek mutable ref object return karta hai jo re-render trigger nahi karta. Ye DOM elements ko directly access karne ya values persist karne ke liye use hota hai across renders.

Q75. useMemo aur useCallback mein kya difference hai?

Ans: useMemo ek computed value ko memoize karta hai (expensive calculations avoid karne ke liye). useCallback ek function reference ko memoize karta hai taaki unnecessary re-renders avoid ho.

Q76. useReducer kab use karte hain?

Ans: useReducer complex state logic ke liye use hota hai jab multiple sub-values ho ya next state previous state par depend kare, Redux jaise reducer pattern follow karta hai.

Q77. Custom hooks kya hote hain?

Ans: Custom hooks 'use' prefix wale reusable functions hain jo built-in hooks ko combine karke logic reuse karne dete hain multiple components ke beech.

Q78. Controlled aur uncontrolled components mein kya farak hai?

Ans: Controlled components ka form data React state se manage hota hai (value + onChange). Uncontrolled components DOM khud apna state maintain karta hai, ref se value access hoti hai.

Q79. React mein keys ki zaroorat kyun hoti hai lists mein?

Ans: Keys React ko batati hain kaunsa item change/add/remove hua hai list re-render ke time, jo efficient reconciliation aur correct component identity maintain karta hai.

Q80. React component lifecycle methods kya hain?

Ans: Mounting (constructor, render, componentDidMount), Updating (shouldComponentUpdate, render, componentDidUpdate), Unmounting (componentWillUnmount) — class components mein use hote hain.

Q81. React.memo() kya karta hai?

Ans: React.memo() ek higher-order component hai jo functional component ko memoize karta hai — agar props change nahi hui toh re-render skip kar deta hai, performance optimization ke liye.

Q82. Prop drilling kya hai aur kaise avoid karein?

Ans: Prop drilling tab hoti hai jab props ko deeply nested components tak pass karna padta hai unused intermediate components ke through. Context API ya state management libraries (Redux, Zustand) se avoid kiya ja sakta hai.

Q83. React Router kya hai?

Ans: React Router ek library hai jo React apps mein client-side routing enable karti hai, single-page applications mein multiple views/pages ka navigation manage karta hai without full page reload.

Q84. Redux kya hai aur kab use karna chahiye?

Ans: Redux ek predictable state management library hai jo single centralized store use karta hai. Large applications mein jab complex, shared state across many components manage karna ho tab useful hai.

Q85. Redux ke core principles kya hain?

Ans: Single source of truth (one store), State is read-only (sirf actions dispatch karke change hota hai), Changes reducers (pure functions) se hote hain.

Q86. Redux mein action, reducer, aur store kya hain?

Ans: Action ek plain object hai jo 'what happened' describe karta hai. Reducer pure function hai jo current state + action leke naya state return karta hai. Store poore app ka state hold karta hai.

Q87. Context API aur Redux mein kya difference hai?

Ans: Context API built-in hai aur simple state sharing ke liye acha hai (theme, auth). Redux zyada powerful hai complex state, middleware, devtools, aur predictable updates ke liye, larger apps mein.

Q88. React Fragments kya hain?

Ans: Fragments (`<>...</>`) multiple elements ko group karte hain bina extra DOM node add kiye, jo cleaner markup deta hai.

Q89. Higher-Order Components (HOC) kya hote hain?

Ans: HOC ek function hai jo component leke ek naya enhanced component return karta hai — reusable component logic ke liye pattern hai, jaise withRouter, connect().

Q90. React mein error boundaries kya hote hain?

Ans: Error boundaries class components hain jo componentDidCatch aur getDerivedStateFromError implement karte hain, child components ke JavaScript errors catch karke fallback UI dikhate hain.

Q91. React.lazy() aur Suspense kya karte hain?

Ans: React.lazy() dynamic import se components ko lazily load karta hai code-splitting ke liye. Suspense loading state (fallback UI) dikhata hai jab tak lazy component load ho raha ho.

Q92. Server-Side Rendering (SSR) kya hai aur React mein kaise hota hai?

Ans: SSR server par HTML generate karke browser ko bhejta hai, jo initial load faster banata hai aur SEO improve karta hai. Next.js jaise frameworks React mein SSR provide karte hain.

Q93. Client-Side Rendering aur SSR mein kya difference hai?

Ans: CSR mein browser JavaScript download karke DOM build karta hai (blank page pehle). SSR mein server pehle se rendered HTML bhejta hai, jo faster First Contentful Paint deta hai.

Q94. React mein state ko lift up karna kya matlab hai?

Ans: State lifting tab hoti hai jab shared state ko common parent component mein move kiya jata hai taaki multiple sibling components use kar sakein via props.

Q95. Portals kya hote hain React mein?

Ans: Portals (ReactDOM.createPortal) child ko parent DOM hierarchy ke bahar render karne deta hai, jaise modals, tooltips jo body ke direct child hote hain.

Q96. PureComponent kya hai?

Ans: PureComponent React.Component ki tarah hai lekin shallow comparison karke shouldComponentUpdate automatically implement karta hai, unnecessary re-renders avoid karne ke liye.

Q97. React mein diffing algorithm kaise kaam karta hai?

Ans: React $O(n)$ heuristic diffing algorithm use karta hai jo same-type elements ko compare karta hai aur keys ke through list items ko efficiently track karta hai, full tree comparison avoid karke.

Q98. Synthetic events kya hain React mein?

Ans: SyntheticEvent React ka cross-browser wrapper hai native DOM events ke around, jo consistent API provide karta hai sabhi browsers mein same behavior ke liye.

Q99. useLayoutEffect aur useEffect mein kya difference hai?

Ans: useLayoutEffect DOM mutations ke baad, browser paint hone se pehle synchronously chalta hai. useEffect asynchronously, paint ke baad chalta hai. Layout measurement ke liye useLayoutEffect use hota hai.

Q100. React mein forms kaise handle karte hain?

Ans: Controlled components ke through, jisme input value React state mein store hoti hai aur onChange handler se update hoti hai, form submission par state read hota hai.

Q101. Code splitting kya hai React mein?

Ans: Code splitting bundle ko chhote chunks mein todta hai jo on-demand load hote hain, jisse initial load time reduce hota hai. React.lazy() aur dynamic import() se achieve hota hai.

Q102. React DevTools kis liye use hote hain?

Ans: React DevTools ek browser extension hai jo component tree, props, state, aur performance ko inspect aur debug karne mein help karta hai.

Q103. Strict Mode kya hai React mein?

Ans: <React.StrictMode> development mein potential problems highlight karta hai — deprecated APIs, unsafe lifecycle methods, side effects double-invoke karke detect karta hai.

Q104. React mein key prop ka galat use karne se kya issue hota hai?

Ans: Index ko key ke roop mein use karna list reorder/insert/delete hone par wrong component state aur unnecessary re-renders cause kar sakta hai — unique stable ID use karna better hai.

Q105. Compound components pattern kya hai?

Ans: Compound components pattern mein multiple components together kaam karte hain implicit shared state share karke (jaise `<Select><Option/></Select>`), flexible aur declarative API deta hai.

Q106. React mein useImperativeHandle kya karta hai?

Ans: useImperativeHandle forwardRef ke sath use hokar parent ko child ke customized instance value/methods expose karta hai, default ref behavior ko override karke.

Q107. Render props pattern kya hai?

Ans: Render props pattern mein component ek function prop leta hai jo React element return karta hai, jisse code logic share hota hai components ke beech.

Q108. useState mein functional update kab use karte hain?

Ans: Jab naya state previous state par depend kare, `setCount(prev => prev + 1)` use karte hain taaki stale closures ke issues avoid ho, especially rapid updates mein.

Q109. React mein hydration kya hai?

Ans: Hydration wo process hai jisme client-side React server-rendered static HTML ko interactive banata hai, event listeners attach karke bina DOM re-create kiye.

Q110. Concurrent rendering React 18 mein kya hai?

Ans: Concurrent rendering React ko multiple state updates ko interrupt/prioritize karne deta hai (transitions), jisse UI responsive rehti hai heavy renders ke bawajood.

Q111. useTransition hook kya karta hai?

Ans: useTransition non-urgent state updates ko mark karta hai as 'transitions', jisse urgent updates (typing) pehle process hon aur UI blocking avoid ho.

Q112. React mein propTypes kya hain?

Ans: PropTypes runtime type-checking provide karte hain component props ke liye, development mein warnings dete hain agar wrong type ki prop pass ho.

Q113. Single Page Application (SPA) kya hoti hai?

Ans: SPA ek web app hai jo single HTML page load karti hai aur JavaScript ke through dynamically content update karti hai bina full page reload ke, better UX ke liye.

Q114. React mein conditional rendering kaise karte hain?

Ans: if-else, ternary operator (`condition ? A : B`), ya logical `&&` operator (`condition && <Component/>`) se conditional rendering ki jaati hai.

Q115. Redux Toolkit kya hai aur kyun use karte hain?

Ans: Redux Toolkit official recommended way hai Redux likhne ka, jo boilerplate reduce karta hai, `createSlice`, `configureStore` jaise utilities deta hai, aur Immer se immutable updates simplify karta hai.

Q116. React Query / TanStack Query kya karta hai?

Ans: React Query server state (API data) ko fetch, cache, sync, aur update karne ke liye library hai, jo loading/error states, caching, refetching automatically manage karta hai.

Q117. useEffect cleanup function kab zaroori hoti hai?

Ans: Cleanup function return karna zaroori hota hai jab subscriptions, timers, ya event listeners set kiye ho, taaki component unmount hone par memory leaks avoid ho.

Q118. React mein key aur id prop mein kya farak hai?

Ans: key React internal use ke liye hai (reconciliation), component ko as prop accessible nahi hota. id ek normal HTML attribute hai jo DOM mein visible hota hai.

Q119. Component composition kya hai?

Ans: Component composition small, reusable components ko combine karke complex UI banane ka approach hai, inheritance ke bajaye children props ya render props use karke.

Q120. React mein forwardRef kya karta hai?

Ans: forwardRef ek component ko ref receive karke usse child DOM element tak forward karne deta hai, especially reusable component libraries banate waqt useful hai.

Q121. Batching kya hai React mein?

Ans: Batching multiple state updates ko single re-render mein combine karta hai performance ke liye. React 18 mein automatic batching hoti hai async operations mein bhi.

Q122. useDeferredValue hook kya karta hai?

Ans: useDeferredValue ek value ka deferred (lagged) version deta hai jo urgent updates ke baad update hota hai, expensive re-renders ko defer karne ke liye.

Node.js

Q123. Node.js kya hai?

Ans: Node.js ek open-source, cross-platform JavaScript runtime environment hai jo Chrome ke V8 engine par built hai, jisse JavaScript server-side run ho sakti hai.

Q124. Node.js single-threaded hone ke bawajood concurrent requests kaise handle karta hai?

Ans: Node.js single-threaded event loop use karta hai lekin I/O operations ko libuv ke through background thread pool mein delegate karta hai, jisse non-blocking asynchronous behavior milta hai.

Q125. Event loop Node.js mein kaise kaam karta hai?

Ans: Event loop different phases mein chalta hai — timers, pending callbacks, poll, check, close callbacks — aur queue mein pending callbacks ko execute karta hai jab call stack empty ho.

Q126. Blocking aur non-blocking code mein kya difference hai?

Ans: Blocking code sequentially execute hota hai, next line tab tak wait karti hai jab tak current operation complete na ho. Non-blocking code operation ko background mein initiate karke aage badh jata hai, callback/promise se result milta hai.

Q127. npm kya hai?

Ans: npm (Node Package Manager) Node.js ka default package manager hai jo dependencies install, manage, aur publish karne ke liye use hota hai.

Q128. package.json aur package-lock.json mein kya farak hai?

Ans: package.json project ki dependencies, scripts, aur metadata define karta hai (version ranges ke sath). package-lock.json exact installed versions lock karta hai reproducible builds ke liye.

Q129. Module.exports aur exports mein kya difference hai?

Ans: module.exports actual object hai jo export hota hai. exports sirf module.exports ka reference hai — agar exports ko reassign kiya (exports = {}) toh original link toot jata hai.

Q130. CommonJS aur ES Modules mein kya farak hai?

Ans: CommonJS require()/module.exports use karta hai, synchronous loading. ES Modules import/export syntax use karte hain, static analysis aur asynchronous loading support karte hain.

Q131. Node.js mein streams kya hote hain?

Ans: Streams data ko chunks mein process karne ka tarika hain, poora data memory mein load kiye bina. Types: Readable, Writable, Duplex, Transform.

Q132. Buffer kya hai Node.js mein?

Ans: Buffer ek class hai jo raw binary data ko handle karti hai, especially file I/O aur network operations mein, kyunki JavaScript mein originally binary data handling nahi thi.

Q133. `process.nextTick()` aur `setImmediate()` mein kya difference hai?

Ans: `process.nextTick()` current operation ke turant baad, event loop ke agle phase se pehle execute hota hai (higher priority). `setImmediate()` check phase mein execute hota hai, poll phase ke baad.

Q134. Middleware kya hai Node.js/Express context mein?

Ans: Middleware functions hain jo request-response cycle mein access rakhte hain (`req`, `res`, `next`) aur request processing, logging, authentication jaise tasks perform karte hain before final handler.

Q135. Node.js mein child processes kaise create karte hain?

Ans: `child_process` module ke methods — `exec()`, `execFile()`, `spawn()`, `fork()` — se new processes create kiye ja sakte hain, CPU-intensive tasks ya external commands run karne ke liye.

Q136. Cluster module kya karta hai Node.js mein?

Ans: Cluster module multi-core systems par multiple Node.js processes (workers) create karta hai jo load share karte hain, application scalability improve karne ke liye.

Q137. Event Emitter kya hai?

Ans: `EventEmitter` ek core class hai jo custom events emit aur listen karne ka pattern implement karti hai (`publish-subscribe`), `.on()` aur `.emit()` methods ke sath.

Q138. REPL kya hai Node.js mein?

Ans: REPL (Read-Eval-Print-Loop) ek interactive shell hai jahan JavaScript code line-by-line execute karke turant result dekh sakte hain, terminal mein 'node' command se start hoti hai.

Q139. Node.js mein environment variables kaise use karte hain?

Ans: `process.env` object se environment variables access hoti hain. `dotenv` package se `.env` file se variables load kiye ja sakte hain.

Q140. Global object kya hota hai Node.js mein?

Ans: global Node.js ka top-level object hai (browser ke window jaisa), jisme globally accessible functions/objects hote hain jaise `process`, `console`, `Buffer`, `require`.

Q141. Node.js mein `__dirname` aur `__filename` kya hain?

Ans: `__dirname` current module ki directory ka absolute path deta hai. `__filename` current module file ka absolute path deta hai.

Q142. Callback function kya hai?

Ans: Callback ek function hai jo dusre function ko argument ke roop mein pass hota hai, aur baad mein execution complete hone par call kiya jata hai, especially asynchronous operations mein.

Q143. Node.js mein error handling kaise karte hain?

Ans: `try-catch` blocks (sync code), `.catch()` (promises), error-first callbacks (`err`, `data`), aur Express mein error-handling middleware se error handling hoti hai.

Q144. Error-first callback pattern kya hai?

Ans: Convention hai jisme callback function ka pehla parameter error object hota hai (null agar no error), doosra actual data — `function(err, data) {...}`.

Q145. fs module kya karta hai?

Ans: fs (File System) module Node.js ka built-in module hai jo files read, write, update, delete, aur watch karne ke liye synchronous aur asynchronous methods provide karta hai.

Q146. Node.js mein API rate limiting kaise implement karte hain?

Ans: express-rate-limit jaisi libraries use karke, ya Redis-based token bucket/sliding window algorithms implement karke requests per time-window limit ki jaati hain.

Q147. libuv kya hai?

Ans: libuv ek C library hai jo Node.js ko asynchronous, event-driven I/O provide karti hai — event loop, thread pool, aur cross-platform async operations implement karti hai.

Q148. Node.js mein memory leaks kaise detect karte hain?

Ans: Chrome DevTools, --inspect flag, heap snapshots, aur tools jaise clinic.js ya node --prof se memory usage analyze karke leaks detect kiye ja sakte hain.

Q149. Worker threads kya hain Node.js mein?

Ans: worker_threads module CPU-intensive JavaScript operations ko separate threads mein run karne deta hai, main event loop ko block kiye bina.

Q150. Node.js applications ko production mein kaise deploy karte hain?

Ans: PM2 (process manager) se process management, environment variables set karke, reverse proxy (Nginx) ke sath, aur cloud platforms (AWS, Heroku, Render, Vercel) par deploy kiya jata hai.

Q151. Node.js mein JWT authentication kaise implement karte hain?

Ans: jsonwebtoken library se login par token generate hota hai (user data + secret sign karke), client token store karta hai aur har request mein Authorization header mein bhejta hai, server verify karta hai.

Q152. REST API kya hai?

Ans: REST (Representational State Transfer) ek architectural style hai APIs design karne ka jo HTTP methods (GET, POST, PUT, DELETE) use karke resources ko manipulate karta hai, stateless communication ke sath.

Q153. Node.js mein CORS kya hai aur kaise handle karte hain?

Ans: CORS (Cross-Origin Resource Sharing) browser security feature hai jo different origin se requests restrict karta hai. cors npm package se Express app mein specific origins allow kiye ja sakte hain.

Q154. HTTP status codes ka significance kya hai?

Ans: 2xx success (200 OK, 201 Created), 3xx redirection, 4xx client errors (400 Bad Request, 401 Unauthorized, 404 Not Found), 5xx server errors (500 Internal Server Error) — API responses ka meaning convey karte hain.

Q155. Node.js mein caching kaise implement karte hain?

Ans: In-memory caching (node-cache), ya external caching (Redis) se frequently accessed data cache kiya jata hai, database load reduce karke response time improve karne ke liye.

Q156. Synchronous aur asynchronous fs methods mein kya difference hai?

Ans: Sync methods (readFileSync) call stack block karte hain jab tak operation complete na ho. Async methods (readFile) callback/promise ke through non-blocking tareeke se result dete hain.

Q157. Node.js mein WebSockets kaise use karte hain?

Ans: socket.io ya ws library se real-time, bidirectional communication client aur server ke beech establish hoti hai, jaise chat apps ya live notifications ke liye.

Q158. Node.js mein API versioning kaise karte hain?

Ans: URL mein version include karke (/api/v1/users), header-based versioning, ya query parameter se different API versions maintain kiye jaate hain.

Q159. bcrypt kya karta hai?

Ans: bcrypt ek library hai jo passwords ko hash aur salt karke securely store karne mein help karti hai, plain text passwords database mein store karne se bachati hai.

Q160. Node.js applications mein logging kaise implement karte hain?

Ans: Libraries jaise winston ya morgan use ki jaati hain structured logging, log levels (info, error, debug), aur file/console output ke liye.

Q161. Node.js mein API testing kaise karte hain?

Ans: Jest, Mocha/Chai, ya Supertest jaisi libraries se unit aur integration tests likhe jaate hain API endpoints validate karne ke liye.

Q162. Node.js mein file upload kaise handle karte hain?

Ans: multer middleware use karke multipart/form-data requests se files ko server mein (ya cloud storage jaise S3) upload kiya jata hai.

Q163. Node.js mein rate limiting aur throttling security ke liye kyun important hai?

Ans: Ye brute-force attacks, DDoS, aur API abuse prevent karte hain by limiting number of requests ek client se ek time window mein.

Q164. npx aur npm mein kya difference hai?

Ans: npm packages install/manage karta hai. npx packages ko without globally installing execute karta hai, one-time use tools run karne ke liye useful hai.

Q165. Node.js mein environment-specific configuration kaise manage karte hain?

Ans: dotenv se .env files (.env.development, .env.production), ya config packages se environment ke basis par different settings load ki jaati hain.

Q166. Node.js ka use case kya hai — kab use karna chahiye?

Ans: Node.js real-time applications (chat, gaming), REST APIs, microservices, aur I/O-heavy applications ke liye ideal hai jahan concurrency zaroori ho, lekin CPU-intensive tasks ke liye kam suitable hai.

Express.js

Q167. Express.js kya hai?

Ans: Express.js ek minimal aur flexible Node.js web application framework hai jo robust features deta hai web aur mobile applications banane ke liye — routing, middleware support, aur HTTP utilities.

Q168. Express mein routing kaise kaam karti hai?

Ans: Routes `app.get()`, `app.post()`, `app.put()`, `app.delete()` jaise methods se define hote hain, jo specific URL path aur HTTP method ke combination par handler function trigger karte hain.

Q169. Middleware Express mein kaise kaam karta hai?

Ans: Middleware functions (`req`, `res`, `next`) hain jo sequentially execute hote hain request ko final route handler tak pahunchne se pehle. `next()` call karke control agle middleware ko pass hota hai.

Q170. `app.use()` kya karta hai?

Ans: `app.use()` middleware functions ko application ke request pipeline mein register karta hai, ya toh globally ya specific path ke liye.

Q171. Express mein error-handling middleware kaise define karte hain?

Ans: Error-handling middleware ke 4 parameters hote hain (`err`, `req`, `res`, `next`) aur ye normal middleware ke baad, sabse last mein define ki jaati hai.

Q172. Router kya hai Express mein?

Ans: `express.Router()` ek mini Express application hai jo modular, mountable route handlers create karne deta hai, related routes ko separate files mein organize karne ke liye.

Q173. Express mein request aur response objects kya hote hain?

Ans: `req` object incoming HTTP request ki details (`params`, `query`, `body`, `headers`) hold karta hai. `res` object server se response bhejne ke methods (`send`, `json`, `status`) provide karta hai.

Q174. `req.params`, `req.query`, aur `req.body` mein kya farak hai?

Ans: `req.params` URL path parameters hain (`/user/:id`). `req.query` URL query string parameters hain (`?sort=asc`). `req.body` request payload hai (`POST/PUT` data), jise parse karne ke liye `body-parser/express.json()` chahiye.

Q175. Express mein `body-parser` ki kya zaroorat hai?

Ans: `body-parser` (ab `express.json()`/`express.urlencoded()` built-in) incoming request body ko parse karke `req.body` mein accessible banata hai, especially JSON aur form data ke liye.

Q176. Express mein static files kaise serve karte hain?

Ans: `express.static()` middleware use karke, jaise `app.use(express.static('public'))`, jo images, CSS, JS files directly serve karta hai.

Q177. Express mein CORS kaise enable karte hain?

Ans: cors npm package install karke `app.use(cors())` se, ya specific origins/methods ke liye options object pass karke enable kiya jata hai.

Q178. RESTful API design principles kya hain?

Ans: Resources ko nouns (URLs) ke roop mein represent karna, appropriate HTTP methods use karna, stateless requests, proper status codes, aur consistent naming conventions follow karna.

Q179. Express mein authentication middleware kaise banate hain?

Ans: Custom middleware likha jata hai jo JWT token verify karta hai (Authorization header se), valid hone par `req.user` set karke `next()` call karta hai, warna 401 response bhejta hai.

Q180. Express mein input validation kaise karte hain?

Ans: Libraries jaise express-validator ya Joi use karke request data ki validation rules define ki jaati hain, invalid data par error response bheja jata hai.

Q181. Express app mein environment-based configuration kaise setup karte hain?

Ans: dotenv se .env file load karke process.env variables use kiye jate hain, development/production ke liye alag configurations set karne ke liye.

Q182. Express mein session management kaise hoti hai?

Ans: express-session middleware se server-side sessions manage hoti hain, ya JWT-based stateless authentication use ki jaati hai scalability ke liye.

Q183. Express mein file upload kaise implement karte hain?

Ans: multer middleware se multipart/form-data handle hota hai, files disk ya memory mein store hoti hain, aur cloud storage (S3) mein bhi upload ki ja sakti hain.

Q184. app.listen() kya karta hai?

Ans: `app.listen(port, callback)` server ko specified port par HTTP requests accept karne ke liye start karta hai.

Q185. Express mein route parameters kaise define karte hain?

Ans: Colon syntax se — `app.get('/users/:id', (req, res) => {...})` — jahan `req.params.id` se value access hoti hai.

Q186. Express mein multiple middleware ek route par kaise chain karte hain?

Ans: Middleware functions ko comma-separated array ke roop mein pass kiya jata hai route definition mein, jo order mein execute hote hain.

Q187. Express mein API response ko standardize kaise karte hain?

Ans: Consistent structure use karke (success flag, data, message, error), aur custom response wrapper functions/middleware banakar.

Q188. Express mein logging kaise implement karte hain?

Ans: morgan middleware HTTP request logging ke liye use hota hai, jo method, URL, status code, response time log karta hai.

Q189. Helmet.js kya karta hai Express mein?

Ans: Helmet security-related HTTP headers set karta hai (XSS protection, content security policy, etc.) taaki common web vulnerabilities se protection mile.

Q190. Express mein rate limiting kaise setup karte hain?

Ans: express-rate-limit middleware se IP-based request limits set kiye jaate hain time window ke andar, brute-force attacks prevent karne ke liye.

Q191. Express mein 404 aur global error handling kaise karte hain?

Ans: Sabse last mein ek catch-all middleware (app.use) 404 handle karta hai, aur error-handling middleware (4 args wala) uncaught errors catch karke proper response deta hai.

Q192. Express mein async route handlers mein errors kaise catch karte hain?

Ans: try-catch blocks manually use karke, ya express-async-errors package/wrapper functions se automatically async errors ko error-handling middleware tak propagate kiya jata hai.

Q193. Express mein API documentation kaise generate karte hain?

Ans: Swagger/OpenAPI (swagger-jsdoc + swagger-ui-express) se interactive API documentation automatically generate ki jaati hai code comments se.

Q194. Express mein pagination kaise implement karte hain?

Ans: Query parameters (page, limit) use karke database query mein skip() aur limit() apply kiya jata hai, aur response mein total count/pages include kiya jata hai.

Q195. Express app mein compression kaise enable karte hain?

Ans: compression middleware use karke response bodies ko gzip compress kiya jata hai, jo bandwidth reduce karta hai aur load time improve karta hai.

Q196. Express mein template engines kaise use karte hain?

Ans: app.set('view engine', 'ejs') jaise setup se EJS, Pug, ya Handlebars server-side rendered HTML pages generate karne ke liye use hote hain.

Q197. Express mein API testing kaise karte hain?

Ans: Supertest library se HTTP requests simulate karke endpoints test kiye jaate hain, Jest/Mocha ke sath integration testing framework banate hain.

Q198. Express mein nested routers kaise use karte hain?

Ans: Router instances ko doosre routers mein mount karke (router.use('/sub', subRouter)), modular aur hierarchical route structure banayi jaati hai.

Q199. next() function ka role kya hai middleware mein?

Ans: next() call karne se control flow agle middleware function ya route handler ko pass hota hai. Agar next() na call ho toh request-response cycle wahin ruk jata hai.

Q200. Express mein request timeout kaise handle karte hain?

Ans: connect-timeout middleware, ya custom timeout logic se long-running requests ko specific time ke baad terminate kiya jata hai.

Q201. Express mein API keys/authentication headers kaise validate karte hain?

Ans: Custom middleware jo req.headers se API key nikaal kar database/config mein match karta hai, valid hone par request aage badhne deta hai.

Q202. Express mein trailing slash routes kaise handle hoti hain?

Ans: Express by default 'strict routing' disabled rakhta hai, matlab /users aur /users/ same route treat hote hain, unless strict mode enable kiya jaye.

Q203. Express mein API response caching kaise implement karte hain?

Ans: Headers jaise Cache-Control set karke, ya Redis se response data cache karke repeated requests ke liye database hits reduce kiye jaate hain.

Q204. Express vs Koa vs Fastify mein kya difference hai?

Ans: Express sabse popular aur mature hai with large ecosystem. Koa lightweight hai async/await-first design ke sath. Fastify performance-focused hai, faster JSON serialization ke sath.

Q205. Express mein CSRF protection kaise implement karte hain?

Ans: csrf jaisi middleware se CSRF tokens generate/verify kiye jaate hain forms mein, cross-site request forgery attacks se bachne ke liye.

MongoDB & Mongoose

Q206. MongoDB kya hai?

Ans: MongoDB ek NoSQL, document-oriented database hai jo data ko flexible, JSON-like BSON documents mein store karta hai, fixed schema ki zaroorat nahi hoti.

Q207. SQL aur NoSQL databases mein kya farak hai?

Ans: SQL databases relational hain, fixed schema, tables/rows/columns mein data store hota hai (MySQL, PostgreSQL). NoSQL databases flexible schema, document/key-value/graph format mein data store karte hain (MongoDB, Redis).

Q208. MongoDB mein collection aur document kya hain?

Ans: Collection MongoDB mein SQL table jaisa hai, jisme documents (rows jaisa) store hote hain. Document ek JSON-like BSON object hai jisme fields aur values hote hain.

Q209. MongoDB mein _id field kya hai?

Ans: _id har document ka unique identifier hai, MongoDB automatically generate karta hai (ObjectId) agar explicitly na diya jaye, primary key ki tarah kaam karta hai.

Q210. BSON kya hai?

Ans: BSON (Binary JSON) MongoDB ka binary-encoded serialization format hai JSON documents store karne ke liye, jo additional data types (Date, ObjectId, Binary) support karta hai.

Q211. Mongoose kya hai?

Ans: Mongoose ek Object Data Modeling (ODM) library hai Node.js ke liye jo MongoDB ke sath schema-based solution provide karta hai — validation, casting, business logic hooks ke sath.

Q212. Mongoose mein Schema aur Model mein kya difference hai?

Ans: Schema document ki structure aur validation rules define karta hai. Model Schema se banaya gaya constructor hai jo actual database operations (CRUD) perform karta hai.

Q213. MongoDB mein CRUD operations kya hain?

Ans: Create (insertOne/insertMany), Read (find/findOne), Update (updateOne/updateMany), Delete (deleteOne/deleteMany) — basic database operations hain.

Q214. Mongoose mein find(), findOne(), aur findById() mein kya farak hai?

Ans: find() saare matching documents ka array return karta hai. findOne() sirf pehla matching document return karta hai. findById() _id ke basis par single document dhundta hai.

Q215. MongoDB mein indexing kya hai aur kyun important hai?

Ans: Index ek data structure hai jo query performance improve karta hai fields par fast lookup enable karke, bina indexing ke MongoDB ko full collection scan karna padta hai.

Q216. MongoDB mein aggregation pipeline kya hai?

Ans: Aggregation pipeline documents ko multiple stages (match, group, sort, project, etc.) se process karta hai, data transformation aur complex analytics queries ke liye use hota hai.

Q217. \$match aur \$group stages kya karte hain aggregation mein?

Ans: \$match documents ko filter karta hai (WHERE clause jaisa). \$group documents ko specified fields ke basis par group karke aggregate values (sum, avg, count) calculate karta hai.

Q218. MongoDB mein embedding aur referencing mein kya difference hai?

Ans: Embedding related data ko same document ke andar nest karta hai (denormalization, fast reads). Referencing ObjectId ke through related documents ko link karta hai (normalization, relational-like).

Q219. Kab embedding use karni chahiye aur kab referencing?

Ans: Embedding tab use karo jab data closely related ho aur saath query hota ho (one-to-few). Referencing tab better hai jab data large ho, frequently update hota ho, ya many-to-many relationship ho.

Q220. Mongoose Schema mein validation kaise define karte hain?

Ans: Schema fields mein required, min, max, minlength, maxlength, match (regex), aur custom validator functions define karke validation rules set ki ja sakti hain.

Q221. Mongoose middleware (hooks) kya hain?

Ans: Pre aur post hooks hain jo schema methods (save, remove, find, etc.) ke pehle ya baad execute hote hain, jaise password hashing pre-save hook mein karte hain.

Q222. MongoDB mein populate() kya karta hai Mongoose mein?

Ans: populate() referenced document (ObjectId) ki jagah actual document data ko automatically fetch aur replace karta hai, joins jaisa behavior deta hai.

Q223. MongoDB transactions kya hain?

Ans: Transactions multiple operations ko atomic unit mein group karte hain — sab succeed hote hain ya sab fail hote hain, data consistency maintain karne ke liye (multi-document ACID operations).

Q224. MongoDB mein sharding kya hai?

Ans: Sharding ek method hai large datasets ko multiple servers mein horizontally distribute karne ka, jisse database horizontally scale ho sake.

Q225. MongoDB mein replication kya hai?

Ans: Replication data ko multiple servers (replica set) par copy karta hai high availability aur data redundancy ke liye — ek primary aur multiple secondary nodes hote hain.

Q226. MongoDB mein primary aur secondary nodes kya hain?

Ans: Primary node saari writes accept karta hai. Secondary nodes primary se data replicate karte hain aur read operations serve kar sakte hain, primary fail hone par election se naya primary select hota hai.

Q227. Mongoose mein virtual fields kya hain?

Ans: Virtual fields document properties hain jo database mein store nahi hoti lekin computed value provide karti hain (jaise fullName firstName+lastName se), get/set ke through define hote hain.

Q228. MongoDB mein \$lookup kya karta hai?

Ans: \$lookup aggregation stage hai jo left outer join perform karta hai ek collection ko doosri collection ke sath, relational join jaisa behavior deta hai.

Q229. MongoDB mein upsert kya hai?

Ans: Upsert (update + insert) ek operation hai jo agar matching document mile toh update karta hai, warna naya document insert kar deta hai.

Q230. Mongoose mein pre-save hook password hashing ke liye kaise use karte hain?

Ans: schema.pre('save', function(next) {...}) mein bcrypt.hash() se password ko hash karke this.password mein assign kiya jata hai, phir next() call kiya jata hai save proceed karne ke liye.

Q231. MongoDB Atlas kya hai?

Ans: MongoDB Atlas ek fully managed cloud database service hai jo automated backups, scaling, monitoring, aur security features provide karta hai without manual server management.

Q232. MongoDB mein compound index kya hai?

Ans: Compound index multiple fields par ek saath index create karta hai, queries ko optimize karta hai jo un multiple fields ko combine karke filter/sort karti hain.

Q233. Explain() query MongoDB mein kya karta hai?

Ans: explain() query execution plan dikhata hai — kitne documents scan hue, index use hua ya nahi, execution time — performance debugging ke liye useful hai.

Q234. MongoDB mein capped collections kya hain?

Ans: Capped collections fixed-size collections hain jo insertion order maintain karti hain aur size limit reach hone par oldest documents automatically overwrite ho jate hain, logging use cases ke liye.

Q235. Mongoose mein schema types kaun kaun se hain?

Ans: String, Number, Date, Buffer, Boolean, Mixed, ObjectId, Array, Map, Decimal128 — ye common Mongoose SchemaTypes hain.

Q236. MongoDB query operators kaun kaun se common hain?

Ans: \$eq, \$ne, \$gt, \$gte, \$lt, \$lte, \$in, \$nin, \$and, \$or, \$not, \$exists, \$regex — comparison aur logical query operators hain.

Q237. MongoDB mein \$set aur \$unset update operators kya karte hain?

Ans: \$set specific field ki value update/add karta hai. \$unset specific field ko document se remove kar deta hai.

Q238. MongoDB mein one-to-many relationship kaise model karte hain?

Ans: Embedding (agar 'many' side chhota ho, jaise blog post ke comments) ya referencing (agar 'many' side bada/independent ho, jaise user ke orders) se model kiya jata hai.

Q239. Mongoose mein lean() query kya karti hai?

Ans: .lean() query ko plain JavaScript objects return karne deta hai instead of full Mongoose documents, jo performance improve karta hai jab document methods ki zaroorat na ho.

Q240. MongoDB mein TTL index kya hai?

Ans: TTL (Time-To-Live) index documents ko automatically ek specified time ke baad delete kar deta hai, jaise sessions ya temporary data expire karne ke liye.

Q241. MongoDB mein text search kaise karte hain?

Ans: Text index create karke \$text operator aur \$search se full-text search queries run ki jaati hain string fields par.

Q242. Mongoose mein schema options timestamps kya karta hai?

Ans: { timestamps: true } option automatically createdAt aur updatedAt fields add karta hai document mein, jab bhi create/update ho tab automatically manage hote hain.

Q243. MongoDB mein ACID properties kaise apply hoti hain?

Ans: Single document operations hamesha atomic hoti hain. MongoDB 4.0+ mein multi-document transactions bhi ACID compliance provide karte hain replica sets mein.

Q244. Mongoose mein instance methods aur static methods mein kya difference hai?

Ans: Instance methods (schema.methods) individual documents par call hote hain. Static methods (schema.statics) model ke level par call hote hain, collection-wide operations ke liye.

Q245. MongoDB mein \$project stage kya karta hai aggregation mein?

Ans: \$project output documents ki shape control karta hai — specific fields include/exclude karta hai ya naye computed fields create karta hai.

Q246. MongoDB mein normalization vs denormalization ka trade-off kya hai?

Ans: Normalization data duplication reduce karta hai lekin joins/lookups zyada karne padte hain. Denormalization (embedding) read performance improve karta hai lekin data duplication aur update complexity badhati hai.

Q247. MongoDB mein findOneAndUpdate() kya karta hai?

Ans: Document ko find karke update karta hai aur (options ke basis par) original ya updated document return karta hai, single atomic operation mein.

Q248. Mongoose validation error kaise handle karte hain?

Ans: try-catch ya .catch() se ValidationError object catch kiya jata hai, jisme har field ki specific error message hoti hai errors object mein.

Q249. MongoDB mein database connection pooling kya hai?

Ans: Connection pooling multiple database connections ko reuse karta hai instead of creating new connection har request ke liye, performance improve karta hai. Mongoose default se pooling handle karta hai.

Q250. MongoDB mein schema-less hone ka fayda aur nuksan kya hai?

Ans: Fayda: flexibility, rapid development, evolving data structures easily handle hoti hain. Nuksan: data consistency ensure karna application layer ki responsibility ban jati hai.

Q251. MongoDB mein \$push aur \$pull operators kya karte hain?

Ans: \$push array field mein new element add karta hai. \$pull array se specific condition match karne wale elements remove karta hai.

Q252. Mongoose discriminators kya hain?

Ans: Discriminators ek schema inheritance mechanism hain jisse ek base model ke multiple variants (sub-schemas) same collection mein different behavior ke sath store kiye ja sakte hain.

Q253. MongoDB mein read concern aur write concern kya hain?

Ans: Read concern batata hai kitna consistent data read karna hai (majority, local). Write concern batata hai write operation ko acknowledge karne ke liye kitne nodes confirm karne chahiye.

Q254. MongoDB Compass kya hai?

Ans: MongoDB Compass ek GUI tool hai MongoDB databases ko visually explore, query, aur manage karne ke liye, bina command line use kiye.

Q255. MongoDB mein geospatial queries kya hain?

Ans: Geospatial queries location-based data (coordinates) par operate karte hain — nearby places dhundhna, area ke andar points check karna — 2dsphere index ke sath.

Q256. Mongoose mein select() method kya karta hai?

Ans: select() specific fields ko include ya exclude karta hai query result mein, jaise .select('name email -password').

Q257. MongoDB mein database backup kaise lete hain?

Ans: mongodump aur mongorestore command-line tools se, ya MongoDB Atlas ke automated backup feature se database backups liye/restore kiye jaate hain.

Q258. MongoDB mein multikey index kya hai?

Ans: Multikey index array fields ke liye automatically create hota hai, jisse array ke har individual element par efficient query ki ja sakti hai.

MERN Integration, Auth, Deployment & System Design

Q259. MERN stack kya hai?

Ans: MERN ek full-stack JavaScript technology stack hai jisme MongoDB (database), Express.js (backend framework), React (frontend library), aur Node.js (runtime) شامل hain — end-to-end JavaScript development possible banata hai.

Q260. MERN stack mein data flow kaise hota hai?

Ans: React frontend se API request Express server ko jati hai (Node.js par), jo MongoDB se data fetch/manipulate karta hai Mongoose ke through, aur JSON response wapas React ko bheja jata hai.

Q261. Frontend aur backend ko MERN mein kaise connect karte hain?

Ans: React app Axios ya fetch API se Express backend ke REST endpoints ko HTTP requests bhejta hai. Development mein proxy setup (package.json mein) ya CORS configuration se cross-origin issues handle hote hain.

Q262. MERN app mein authentication flow kaise design karte hain?

Ans: User login karta hai, Express server credentials verify karke JWT token generate karta hai, React token ko localStorage/cookie mein store karta hai aur har protected request mein Authorization header mein bhejta hai.

Q263. JWT (JSON Web Token) kya hai?

Ans: JWT ek compact, self-contained token hai jisme header, payload, aur signature hote hain, user authentication information securely transmit karne ke liye use hota hai bina server-side sessions ke.

Q264. JWT ko localStorage mein store karna secure hai ya cookies mein?

Ans: localStorage XSS attacks ke liye vulnerable hota hai. HttpOnly cookies zyada secure hain kyunki JavaScript se access nahi ho sakte, lekin CSRF protection ki zaroorat hoti hai.

Q265. Access token aur refresh token mein kya difference hai?

Ans: Access token short-lived hota hai aur API requests authenticate karta hai. Refresh token long-lived hota hai aur naya access token generate karne ke liye use hota hai bina user ko dobara login kiye.

Q266. MERN app mein protected routes React mein kaise banate hain?

Ans: PrivateRoute/ProtectedRoute wrapper component banaya jata hai jo authentication status check karta hai (Context/Redux se) — agar authenticated nahi toh login page par redirect karta hai.

Q267. MERN stack app deployment ka common approach kya hai?

Ans: Frontend (React build) ko Vercel/Netlify par, backend (Node/Express) ko Render/Railway/Heroku par, aur database MongoDB Atlas par deploy kiya jata hai, ya poora app

single server (AWS EC2) par bhi deploy ho sakta hai.

Q268. React app ko production ke liye build kaise karte hain?

Ans: npm run build command se optimized, minified static files (HTML, CSS, JS bundle) generate hote hain jo deployment ke liye ready hote hain.

Q269. MERN app mein environment variables kaise manage karte hain frontend aur backend dono mein?

Ans: Backend mein .env file + dotenv package. Frontend (Create React App) mein REACT_APP_ prefix wale variables .env file mein, jo build time par embed hote hain.

Q270. MERN stack mein state management ke liye kya options hain?

Ans: Local component state (useState), Context API, Redux/Redux Toolkit, Zustand, ya server state ke liye React Query/TanStack Query use kiye ja sakte hain.

Q271. Full-stack app mein API error handling kaise design karte hain?

Ans: Backend consistent error response format bhejta hai (status code + message), frontend try-catch/error boundaries se errors catch karke user-friendly messages dikhata hai.

Q272. MERN app mein file upload flow kaise kaam karta hai?

Ans: React form multipart/form-data ke sath file ko Express backend bhejta hai (multer middleware handle karta hai), jo file ko disk/cloud storage (S3, Cloudinary) mein save karke URL database mein store karta hai.

Q273. MERN stack app mein pagination frontend-backend mein kaise implement karte hain?

Ans: Backend query parameters (page, limit) accept karke skip/limit se data return karta hai with total count. Frontend page number track karke API calls karta hai aur pagination UI dikhata hai.

Q274. MERN app mein real-time features kaise add karte hain?

Ans: Socket.io library Express server aur React client dono mein integrate karke real-time bidirectional communication (chat, live notifications) implement ki jaati hai.

Q275. MERN stack mein testing strategy kya hoti hai?

Ans: Backend: Jest/Mocha for unit tests, Supertest for API testing. Frontend: React Testing Library/Jest for component testing. End-to-end: Cypress ya Playwright.

Q276. Monolithic architecture aur microservices mein kya difference hai?

Ans: Monolithic mein poora application single codebase/deployment unit hota hai. Microservices mein application ko chhote, independently deployable services mein todte hain jo APIs se communicate karte hain.

Q277. Load balancing kya hai?

Ans: Load balancing incoming traffic ko multiple servers mein distribute karta hai, jisse koi single server overload na ho aur high availability/performance mile.

Q278. Horizontal aur vertical scaling mein kya farak hai?

Ans: Vertical scaling ek server ki resources (CPU, RAM) badhata hai. Horizontal scaling multiple servers add karta hai load ko distribute karne ke liye.

Q279. API rate limiting kyun zaroori hai?

Ans: Rate limiting server ko abuse, DDoS attacks, aur excessive resource usage se protect karta hai, fair usage ensure karke.

Q280. MERN stack mein performance optimization ke liye kya techniques use karte hain?

Ans: Frontend: code splitting, lazy loading, memoization. Backend: caching (Redis), database indexing, compression, CDN use karna.

Q281. CI/CD pipeline kya hai?

Ans: Continuous Integration/Continuous Deployment ek automated process hai jo code changes ko test, build, aur deploy karta hai automatically, GitHub Actions/Jenkins jaise tools se.

Q282. Docker kya hai aur MERN app mein kaise use hota hai?

Ans: Docker application aur uski dependencies ko containers mein package karta hai, consistent environment ensure karta hai. MERN app ke different services (frontend, backend, DB) alag containers mein run ho sakte hain.

Q283. MERN app mein security best practices kya hain?

Ans: Input validation/sanitization, HTTPS use karna, environment variables mein secrets rakhna, helmet.js, rate limiting, password hashing (bcrypt), aur CORS properly configure karna.

Q284. SQL injection aur NoSQL injection attacks kya hain?

Ans: In attacks mein malicious input ke through database queries manipulate ki jaati hain. NoSQL mein operators (\$where, \$ne) inject karke query logic change ki ja sakti hai — input sanitization se prevent hoti hai.

Q285. XSS (Cross-Site Scripting) attack kya hai aur kaise prevent karein?

Ans: XSS mein attacker malicious scripts inject karta hai webpage mein jo doosre users ke browser mein execute hote hain. Input sanitization, output encoding, aur Content Security Policy se prevent kiya jata hai.

Q286. CSRF attack kya hai?

Ans: Cross-Site Request Forgery mein attacker authenticated user ko unknowingly unwanted actions perform karwata hai. CSRF tokens aur SameSite cookies se protection milti hai.

Q287. Environment variables production mein securely kaise manage karte hain?

Ans: Cloud provider ke secret management tools (AWS Secrets Manager), ya platform-specific environment variable settings (Vercel, Render dashboard) use kiye jaate hain, .env files ko git mein commit nahi karte.

Q288. MERN app mein logging aur monitoring kaise setup karte hain?

Ans: winston/morgan se application logs, aur tools jaise Sentry (error tracking), New Relic/Datadog (performance monitoring) production monitoring ke liye use hote hain.

Q289. REST API aur GraphQL mein kya difference hai?

Ans: REST mein multiple fixed endpoints hote hain, over/under-fetching ho sakti hai. GraphQL single endpoint se client exact data specify karke fetch kar sakta hai, flexible queries ke sath.

Q290. MERN app mein database schema design karte waqt kya consider karna chahiye?

Ans: Query patterns, read/write ratio, data relationships (embed vs reference), scalability requirements, aur data consistency needs consider karni chahiye.

Q291. Idempotency kya hai APIs mein?

Ans: Idempotent operation multiple baar call karne par same result deta hai jaisa ek baar call karne par (GET, PUT, DELETE idempotent hain; POST generally nahi).

Q292. MERN stack app mein SEO kaise improve karte hain?

Ans: Server-side rendering (Next.js), meta tags management (react-helmet), sitemap generation, aur proper semantic HTML use karke SEO improve kiya jata hai.

Q293. Webpack kya hai aur React mein kya role hai?

Ans: Webpack ek module bundler hai jo JavaScript files, CSS, images ko optimize karke ek ya zyada bundles mein combine karta hai. Create React App internally Webpack use karta hai.

Q294. Babel kya hai?

Ans: Babel ek JavaScript compiler/transpiler hai jo modern JS (ES6+) aur JSX ko older, browser-compatible JavaScript mein convert karta hai.

Q295. MERN app mein version control best practices kya hain?

Ans: Meaningful commit messages, feature branches, .gitignore mein node_modules/.env include karna, pull requests aur code review process follow karna.

Q296. Full-stack developer interview mein system design kaise approach karein?

Ans: Requirements clarify karo, high-level architecture design karo (client-server-database), scalability/bottlenecks discuss karo, trade-offs explain karo, aur incrementally detail add karo.

Q297. MERN stack ka kya advantage hai?

Ans: Single language (JavaScript) full-stack mein use hoti hai, jo development faster aur consistent banata hai, code reuse aur JSON data flow seamlessly poore stack mein possible hota hai.

Q298. MERN stack ki kya limitations hain?

Ans: Single-threaded Node.js CPU-intensive tasks ke liye ideal nahi. MongoDB NoSQL hone ki wajah se complex relational data/transactions ke liye SQL databases zyada suited ho sakte hain.

Q299. MERN app mein API documentation share karne ke best practices kya hain?

Ans: Swagger/OpenAPI se interactive docs generate karna, Postman collections share karna, aur README mein endpoints, request/response examples, aur auth requirements clearly likhna.

Q300. MERN stack interview mein commonly puchha jane wala coding round kya hota hai?

Ans: Generally ek CRUD REST API banane ko kaha jata hai (Express + MongoDB), saath mein ek React component/UI banane ko, jisme state management, API integration, aur error handling dikhani hoti hai.